UTN FRLP

Desarrollo de Software 2026

Armado de Backlog + JS/TS Moderno

Clase #05 - 27 de abril del 2026

Matías Corti - Martina García Améndola

Temas

1º Parte: Armado de Backlog (Fase 3)

- Por qué un backlog
- Épicas, features, historias de usuario
- Priorización y estimación
- Criterios de aceptación
- 📄 Material complementario: Procedimiento para Armado de Backlog (Fase 3)

2ª Parte: JS/TS Moderno

- Módulos, arrow functions, destructuring
- Spread / rest
- Promises, async/await, fetch
- Práctica: consumir una API pública
- 📄 Material complementario: Apunte JavaScript y TypeScript

1º Parte

Armado de Backlog (Fase 3)

¿Por qué un backlog?

Tenemos:

- 18 requisitos funcionales
- 12 reglas de negocio
- Modelo de dominio con 8 entidades
- Un cliente que quiere todo "para ayer"
- Y 7 personas en el equipo durante 7 meses.

Una lista plana de tareas no alcanza. Necesitamos:

- **Priorizar** ¿qué hacemos primero?
- **Dividir** ¿cómo lo dividimos en partes?
- **Estimar** ¿cuánto nos va a llevar cada uno?
- **Conectar** ¿qué valor le da esto al usuario?

Eso es el backlog: una lista ordenada, viva y priorizada.

Product Backlog

Lista única, ordenada y viva de todo el trabajo que el sistema necesita.

Características

- Ordenada -> lo más valioso arriba
- Estimada -> sabemos cuánto pesa cada cosa
- Visible -> todo el equipo y el cliente la ven
- Viva -> cambia con cada conversación, cada entrega

NO es:

- Una lista de tareas técnicas
- Una to-do list congelada
- "Todo lo que el cliente pidió alguna vez"

Épica



Feature



Historia

Concepto	Qué es	Ejemplo (Biblioteca)
Épica	Un bloque grande de funcionalidad.	Gestión de Préstamos
Feature	Una funcionalidad concreta dentro de una épica.	Proceso de Devolución
Historia de Usuario	Una descripción breve y sencilla de una funcionalidad, escrita desde la perspectiva del usuario final.	Como usuario, quiero devolver un libro escaneando su código.

Relación

```
Épica
    └── Feature
        └── Historia de Usuario
        └── Historia de Usuario
    └── Feature
        └── Historia de Usuario
```

Historia de Usuario

Formato simple

```
Como   [rol],
quiero [objetivo],
para   [beneficio].
```

Ejemplo

```
Como   usuario,
quiero buscar libros por título,
para   encontrar rápidamente el que
necesito.
```

Parte	Pregunta que responde
Como [rol]	¿Quién lo necesita?
quiero [objetivo]	¿Qué quiere hacer?
para [beneficio]	¿Para qué le sirve? (valor)

Criterios de Aceptación

Formato

```
Given/Dado    [contexto / precondición],
When/Cuándo   [acción / evento],
Then/Entonces [resultado esperado].
```

Ejemplos

```
Dado que estoy en el catálogo,
cuando escribo un título en el buscador,
entonces se muestran los libros que
coinciden.
--
Dado que busco un título que no existe,
cuando presiono buscar,
entonces se muestra "No se encontraron
resultados"
```

- Es la definición de "listo": si los criterios pasan, la historia es correcta.
- Permite testear, manual y automáticamente.
- formato similar al de tests automáticos.

Actividad: Estimaciones

Estimación

Planning Poker

Formen **equipos de ~4 personas** según su proximidad.

Cada uno, cortando una hoja, **arma 8 cartas,** cada una **con uno de con estos valores:**

1, 2, 3, 5, 8, 13, 21 y 34.

Para cada tarea que yo lea:

1) discutan brevemente en equipo (hasta que yo lo indique) y concluyan un resumen.
2) cada uno elije una carta (sin mostrarla). Luego todos la revelan simultáneamente.
3) La persona con la estimación más baja y la persona con la más alta justifican su estimación.
4) Se repite el procedimiento desde el paso 1) hasta alcanzar un consenso.

El objetivo no es acertar un tiempo exacto, sino establecer el tamaño relativo de cada funcionalidad

Como usuario,

quiero buscar libros por título, autor o palabra clave,

para conocer su disponibilidad.

Como usuario registrado,

quiero poder solicitar el préstamo de un libro disponible,

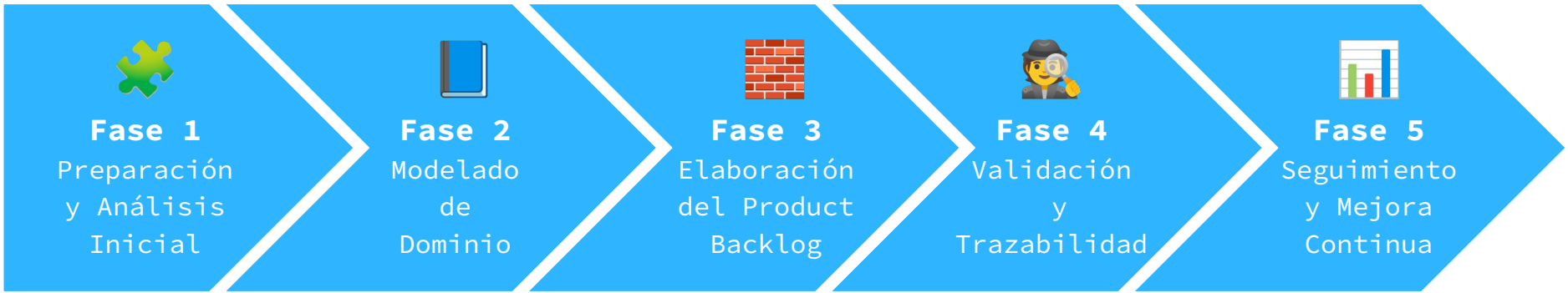
para llevármelo a casa.

Como bibliotecario,

quiero que el sistema calcule automáticamente el valor de una multa por retraso

para informarle al usuario el importe.

Procedimiento Para Armado de Backlog



LINK AL DOCUMENTO

2ª Parte

JS/TS Moderno

¿Por qué Importante?

Apunte de JavaScript y TypeScript

En C03 vimos JS básico.

En C04 vimos TS introductorio.

Hoy es un "puente" antes de React.

Cuando arranquemos con React y se sientan perdidos, la mayoría de las veces el problema tendrá que ver con:

- *No entender cómo se importa un archivo*
- *No saber desestructurar*
- *No haber entendido async/await*
- *No saber qué devuelve fetch*

*Estos temas los seguimos en el **apunte**.*



Práctica

async/await

Fetch



Práctica

async/await

Fetch

- 3 ejercicios con dificultad creciente
- Cada ejercicio **en su carpeta** dentro del repositorio:

```
ds2026-actividades/
└── ejercicios/
    └── c05-async-await-fetch/
        ├── ej1-usuarios/
        ├── ej2-lista-html/
        └── ej3-buscador-libros/
```

- Cada ejercicio terminado = **un commit** (con mensaje descriptivo)
- ⏱️ **En clase:** arrancar desde el Ejercicio 1, llegar hasta donde puedan
- 📅 **Durante la semana:** completar hasta el Ejercicio 3

Ejercicios Prácticos

Ejercicio 1 - Fetch básico tipado
Objetivo: *Que llamen a una API real con async/await y aprendan a tipar lo que viene del servidor en lugar de tratarlo como any.*
API:
JSONPlaceholder (https://jsonplaceholder.typicode.com)
Pasos:
- Crear un proyecto con index.html y app.ts.
- Definir una interface `Usuario` con: id, name, email, phone.
- Escribir una función `async function obtenerUsuarios(): Promise<Usuario[]>` que use fetch y devuelva los usuarios tipados.
- Llamar la función y mostrar nombre y email de cada usuario en la consola.
- Manejar errores con `try/catch`.

Ejercicio 2 — DOM + fetch
Objetivo: *Conectar el resultado del fetch con la pantalla: render dinámico, estado de carga y manejo visual de errores. La estructura mínima de cualquier vista que consume datos.*
Pasos:
- Partir del HTML del Ej 1, cuando se reciben los datos, mostrar la lista de usuarios como una `<ul>` en la página, cada `<li>` con el nombre y el email.
- Mientras carga, mostrar un `<p>Cargando…</p>`. Cuando termina, ocultarlo.
- Si hay error, mostrar un mensaje rojo en la página.

Ejercicio 3 - Buscador de libros
Objetivo: *Trabajar con datos reales (campos opcionales, respuestas anidadas) y eventos del usuario. Acercarse a una mini-funcionalidad completa: input → acción → resultados.*
API:
Open Library (https://openlibrary.org/search.json?q=...)
Pasos:
- Crear una página con: un `<input type="text">`, un `<button>Buscar</button>` y un contenedor `<div id="resultados">`.
- Al hacer click, hacer `fetch` a la URL con el texto del input como parámetro `q`.
- La respuesta tiene un campo docs que es un array. Definir una interface `LibroOL` con title, author_name?, first_publish_year? (los campos pueden faltar, así que opcionales).
- Mostrar los primeros 10 resultados como tarjetas con título, autor (si existe) y año (si existe).
- **Validar**: input vacío → mostrar mensaje de error, no hacer `fetch`.

Actividades de la semana

Actividad individual de la semana

1º instancia hasta el **03/05/2026**
2º instancia: ***10/05/2026*** *(Cierre Bloque 1)*

[AI-C05 - Async/Await + Fetch]
Completar Ejercicios Prácticos

- Completar al menos los **ejercicios 1 al 3**
- Cada ejercicio en su carpeta, cada uno con su commit
- Subir todo al su repositorio ds2026-actividades en GitHub, en un subdirectorio nuevo para esta clase:

  ejercicios/c05-async-await-fetch/

Una vez completada la entrega, escribir en Discord un mensaje en el canal **#actividades**, dentro del Hilo: ***"[AI-C05] — Async/Await + Fetch".***

Actividad grupal de la semana

Hasta el **03/05/2026** (inclusive)

[AG-C05 - Backlog completo del TP (Fase 3)]

Completar Fase 3 del "Procedimiento Para Armado de Backlog"

1. **Completar Fase 3:** Épicas y features (con prioridad y estimación), Historias de Usuario completas (Anexo A), Aplicar Checklist de Calidad (Anexo B) a cada HU
2. Crear en el repositorio, dentro de la carpeta "Documentación", la carpeta: **"03-Backlog"** Subir en ella el/los documentos correspondientes.
3. Actualizar el índice del README.md de la carpeta "Documentación" .
4. Comunicar la finalización en Discord en un mensaje único por equipo, dentro del canal **#actividades**, en el Hilo: ***"[AG-C05] – Backlog completo del TP (Fase 3)"***.